

Design Decisions & Rationale

ProjectAlma

Status: living document · Last updated: 2026-08-31

This document records **why** it is that way: the options that were on the table, the one that was chosen, and what it costs us. Each entry is a lightweight ADR - context, alternatives, decision, consequences, and the signal that should make us revisit it.

The guiding principle across all of these: **this is a take-home-sized codebase that has to read like production**. So the bias is toward (a) getting the **seams** right - interfaces at every external boundary - and (b) not paying infra cost for scale we don't have yet, as long as the swap is cheap and localized.

Index

1. [Two services with Next.js as a backend-for-frontend](#)
 2. [FastAPI + a fully async request path](#)
 3. [PostgreSQL in production, SQLite for dev and tests](#)
 4. [The public form requires no account](#)
 5. [Applicant ↔ lead ownership by email, not a foreign key](#)
 6. [Lead status as an explicit, data-driven state machine](#)
 7. [`REACHED\\_OUT` is reversible](#)
 8. [Append-only `lead\\_events`, written in the same transaction](#)
 9. [One error envelope + typed domain exceptions](#)
 10. [JWT in an httpOnly cookie, no refresh token](#)
 11. [Attorney self-registration behind an invite code](#)
 12. [Email as a background task behind a provider interface](#)
 13. [Pluggable storage and rate-limiting behind ABCs](#)
 14. [Frontend: URL-as-state, server components, no data-fetching library](#)
 15. [Confirm dialogs instead of an undo/toast pattern](#)
-

1. Two services with Next.js as a backend-for-frontend

Context. The brief asks for a FastAPI backend and a Next.js frontend. The open question is how the browser talks to the API and where the session token lives.

Options considered.

Option	Notes
Browser → FastAPI directly, token in <code>localStorage</code>	Simplest wiring. Token readable by any XSS payload; CORS on the hot path; API host is public.
Browser → FastAPI directly, token in a cookie set by FastAPI	Cookie is cross-origin; needs <code>SameSite=None</code> ; <code>Secure</code> + strict CORS; CSRF surface.
Browser → thin Next route handlers → FastAPI; token in an <code>httpOnly</code> cookie set by Next	Token never in JS. All browser traffic same-origin. API host stays private. One extra hop.
Next.js full-stack (route handlers <i>are</i> the backend)	Fewer moving parts, but throws away the FastAPI requirement and the clean Python service/test story.

Decision. Next.js is a **backend-for-frontend**. Every browser call is same-origin to `/api/*` route handlers that attach the JWT from an `httpOnly`, `SameSite=Lax` cookie and forward to FastAPI. Server Components call the API directly server-side.

Consequences.

- XSS can't exfiltrate the token; no CORS on the critical path; the API can be network-private in production.
- FastAPI stays a clean, independently testable service with no view concerns.
- One extra network hop and a small amount of proxy boilerplate per endpoint

(`app/api/*/route.ts`).

- Two deployables to run and version.

Revisit if. The proxy layer starts accumulating real logic (it should stay dumb), or we adopt a gateway/BFF platform that does token custody for us.

2. FastAPI + a fully async request path

Context. The two slowest things this app does are **file uploads** and **database round-trips**. Under load, the failure mode of a sync WSGI app is worker-thread starvation while everyone waits on I/O.

Options considered. Django (batteries, admin, ORM, migrations - but sync by default and heavier than the domain warrants); Flask (minimal, but I'd re-assemble validation + serialization + async myself); **FastAPI** (async-native, Pydantic validation and OpenAPI for free, small surface).

Decision. FastAPI on ASGI, with the async path carried **all the way down**:

async def handlers → async services → SQLAlchemy 2.0 async ORM → asyncpg, boto3 and bcrypt (the two unavoidably-sync pieces) run in a threadpool.

Consequences.

- A slow upload or a slow query parks a coroutine, not a worker.
- Request/response validation and docs are declarative.
- Async SQLAlchemy is less mature; lazy-loading rules are stricter (we use `selectin` eager loading and explicit `flush()` deliberately).
- "async all the way" is a discipline - one sync DB call in the path silently

blocks the loop. **Revisit if.** We need Django's admin for ops, or the team is materially more productive in sync code than the load justifies.

3. PostgreSQL in production, SQLite for dev and tests

Context. Contributors should be able to clone and run with zero infra. CI should be fast. Production needs concurrency, real types, and migrations.

Decision. One `DATABASE_URL`, two dialects. `sqlite+aiosqlite` is the default in `.env.example` and the test backend (in-memory); `postgresql+asyncpg` is the Docker/production target. The ORM and all queries are written to the common subset; Alembic migrations are authored against Postgres.

Consequences.

- `git clone && uvicorn` works with nothing installed. The full 60+ test suite

runs in ~15s with no database container.

- **Dev/prod parity gap.** SQLite doesn't enforce some constraints, has

coarser datetime precision, and different locking. We've already been bitten once - event ordering needed a Python-side default for microsecond precision on SQLite (see `LeadEvent.created_at`).

- CI must *also* run migrations against a real Postgres service to catch

dialect-specific DDL. It does.

Revisit if. The parity gap causes a second production surprise - at that point, Testcontainers-Postgres everywhere is worth the CI minutes.

4. The public form requires no account

Context. The requirement: "a **publicly available** form for a prospect." Later we added an applicant portal. The tension: a portal wants a user identity; the form must not.

Decision. The form is fully anonymous - `POST /leads` has no auth, only rate limiting and upload hardening. Accounts are **optional and additive**: a prospect can submit first, create an account later, and still see their submission (see ADR 5).

Consequences.

- Lowest possible friction on the one flow that's the point of the product.
- An unauthenticated public endpoint that writes to the DB and stores files is

an abuse target. Mitigations in place: fixed-window rate limit per IP, content-type allow-list + magic-byte sniff, size cap with a bounded read, filename sanitization. Mitigations *documented, not built*: CAPTCHA, spam heuristics, virus scanning of uploads.

Revisit if. Abuse volume shows up in the logs - CAPTCHA is the first lever.

5. Applicant ↔ lead ownership by email, not a foreign key

Context. A prospect submits the form (no account), then signs up. The portal at `/my` must show them "their" leads. At submit time there is no user row to point a foreign key at.

Options considered.

Option	Problem
Require an account before submitting	Violates ADR 4 / the brief.
Claim-token link emailed on submission	Best UX-integrity but more moving parts (token table, expiry, email deliverability on the critical path).
Match <code>lead.email</code> to <code>user.email</code> at read time	Zero submit-time cost; works retroactively. Trusts the email as an identifier.

Decision. Ownership is `lead.email == current_user.email` (both lower-cased), resolved in `GET /my/leads`. No FK, no backfill.

Consequences.

- Submit-then-signup "just works," including for leads created before the user existed.

- **Security depends on registration verifying email ownership.** Today

registration does *not* send a verification mail - so this is only sound once it does. This is called out as a required production hardening, not a finished property.

- No `ON DELETE` semantics between users and their leads; the relationship is a runtime join, not a schema guarantee.

Revisit before production. Add email verification on registration - it's the precondition that makes this model safe. A claim-token flow is the stronger alternative if we want ownership to be explicit in the data.

6. Lead status as an explicit, data-driven state machine

Context. The brief has two states and one manual transition. It would be easy to represent this as a boolean or a free-form string and check it ad hoc.

Decision. A `LeadState` enum plus a single transition table (`services/leads.py::_TRANSITIONS: dict[LeadState, set[LeadState]]`). `update_lead_state` is the **only** writer; it validates the move against the table, raises `ConflictError` ($\rightarrow$ 409) on an illegal one, treats a no-op move as a successful 200, and writes the audit event. The API accepts a target state, not an RPC verb like `/reach-out`.

Consequences.

- Adding a state (`IN_REVIEW`, `CLOSED`, ...) is a table edit plus tests - the

enforcement, audit, and error handling are already generic.

- Illegal transitions fail loudly and consistently.
- Slightly more ceremony than a boolean for today's two states. This is a deliberate bet that lead lifecycles always grow.

- A `PATCH` with a target state (rather than a named action) means the client

has to know the valid targets. Acceptable - the milestone label and the available buttons are derived from the same enum.

Revisit if. Transitions ever need side conditions richer than "from-state is allowed" (e.g. role-per-transition, required fields) - then a small rules object per transition beats the set-of-targets table.

7. REACHED_OUT is reversible

Context. Originally `REACHED_OUT` was terminal: `_TRANSITIONS[REACHED_OUT]` was empty and a reverse move returned 409. The brief's wording ("transitions to `REACHED_OUT` when marked manually") reads as one-directional. But an attorney clicking the wrong row's button then needed a DBA to fix it.

Options considered.

Option	Trade-off
Keep terminal; fix mistakes with a manual DB edit	"Pure" model, unusable ops story.

Time-boxed undo (revert allowed for N minutes)	Solves the fat-finger case; adds a clock to the domain logic and a "why is the button gone now" UX question.
Allow `REACHED_OUT → PENDING`, log every move	Simple, symmetric, fully auditable. Loosens the "once contacted, always contacted" invariant.

Decision. Both directions are legal. Reopening clears `reached_out_at` / `reached_out_by_id` and appends a distinct event, `"REACHED_OUT -> PENDING (reopened)"`. Nothing is deleted or overwritten - the forward event stays in the trail (ADR 8), so history is never rewritten, only extended. The UI gates it behind a confirm dialog (ADR 15).

Rationale. Both states are non-destructive, and the audit trail already records **who** and **when** for every move - so the strict "terminal" rule was buying integrity we were already getting from the event log, at the cost of a real operational hazard. The denormalized `reached_out_*` columns are a convenience cache of "the current reached-out fact"; clearing them on reopen keeps them honest, and the event log remains the source of truth for history.

Consequences.

- Mistakes are self-service and leave an honest trail.
- "Reached out" is no longer a monotonic, latching fact - a report that counts

`reached_out` leads is a point-in-time snapshot, not a cumulative total. If we ever need "was this lead **ever** contacted," that's a query over `lead_events`, not a column check.

Revisit if. Compliance requires contact to be a latching fact - then reopen becomes a separate `REOPENED` state rather than a return to `PENDING`, preserving monotonicity while still allowing work to resume.

8. Append-only `lead_events`, written in the same transaction

Context. "Who marked this, and when?" and "what has happened to this lead?" are questions an internal tool will always get asked. `updated_at` alone can't answer them.

Decision. Every lead creation and every state change inserts a row into an append-only `lead_events` table **in the same transaction** as the change itself. Events are never updated or deleted. `created_at` is set in Python (microsecond precision) so same-tick events order deterministically. The denormalized `reached_out_at` / `reached_out_by_id` columns on `leads` are a cache for the common "current status" read; the event log is the system of record for history.

Consequences.

- The trail can't drift from reality - if the state change commits, so does

its event; if it rolls back, neither happens.

- Powers the per-lead timeline and the cross-lead activity feed with no extra

bookkeeping.

- Two writes per transition. Negligible at this volume.
- The table grows unbounded. Fine for years at lead volume; archival is a

future concern, not a current one.

Revisit if. We add many event types and the feed queries need their own indexes or a read model.

9. One error envelope + typed domain exceptions

Context. A frontend that has to special-case FastAPI's default validation shape, our own 4xx shapes, and unhandled 500s will grow bugs at every boundary.

Decision. Every non-2xx response has exactly one shape: `{"error": {"code, message, details, request_id}}`. The service layer raises typed exceptions (`NotFoundError`, `ConflictError`, `RateLimitedError`, ...) that know nothing about HTTP; a single set of handlers in `main.py` maps them to status + envelope. Pydantic validation errors and uncaught exceptions are funnelled through the same formatter. 429 carries `Retry-After`; every response carries `X-Request-ID`.

Consequences.

- The frontend has one error path. Support can correlate a user report to logs by request id.
- Services stay HTTP-agnostic and unit-testable.
- A small taxonomy to maintain, and the discipline to raise the typed error

rather than a bare `HTTPException` in a route.

10. JWT in an httpOnly cookie, no refresh token

Context. Sessions for a small internal tool + an applicant portal.

Decision. Password login issues a single HS256 access token (claims `sub`, `iat`, `nbf`, `exp`, `iss`, `type`; decode verifies all of them). It's stored in an `httpOnly` cookie by the Next login handler. No refresh token, no server-side session store. TTL is a config knob (default long, for demo comfort).

Consequences.

- Stateless verification, nothing to store or invalidate-check on the hot path, XSS-safe storage (ADR 1).
- **No revocation.** A stolen token is valid until it expires; "log out

everywhere" and "disable this user immediately" aren't possible without a denylist. `is_active` is checked on every request, which covers the disable-user case at DB latency but not instantly-at-the-edge.

- Long TTL for demo convenience is the wrong default for production - noted in

the hardening list. The real answer is a short access TTL + refresh rotation + silent renew.

Revisit before production. Short TTL + refresh tokens; consider a Redis denylist keyed by `jti` if immediate revocation is required.

11. Attorney self-registration behind an invite code

Context. Attorneys see **all** lead PII. Applicant registration needs to be open (people tracking their own case). The same `/auth/register` endpoint serves both.

Decision. `register` takes a `role`. `applicant` is open. `attorney` additionally requires a shared invite code (`ATTORNEY_SIGNUP_CODE`, compared with `secrets.compare_digest`), is separately rate-limited, can be switched off entirely (`ATTORNEY_SIGNUP_ENABLED=false`), and `Settings._guard_production` **refuses to boot** if the code is still the default welcome.

Consequences.

- A working self-serve path for the demo without leaving internal access wide

open. Defense in depth: code + rate limit + kill switch + boot guard.

- A shared secret is not real access control - anyone with the code is an

attorney. It's a speed bump, explicitly.

Revisit before production. Replace with admin-issued invitations (single-use, expiring, emailed) or SSO/SCIM against the firm's directory. The `role` column and the `get_current_attorney` dependency are already the enforcement points.

12. Email as a background task behind a provider interface

Context. Two emails per submission (prospect + attorney). Email must never make the submission slow or fail, and dev/CI must run without network.

Decision. An `EmailProvider` ABC with `ResendEmailProvider` (used when `RESEND_API_KEY` is set) and `ConsoleEmailProvider` (the default - logs the message). Messages are pure (`subject`, `html`, `text`) builder functions. Dispatch is via FastAPI `BackgroundTasks` - after the response is sent; the provider swallows and logs its own errors.

Consequences.

- Submission latency and success are independent of the mail provider. The

whole flow is demonstrable offline. Swapping providers is one class.

- **`BackgroundTasks` is in-process and not durable.** If the worker dies

between response and send, that email is lost - no retry, no dead-letter. This is acceptable at low volume and fails safe (the lead is still recorded), but it's the first thing to outgrow.

Revisit if. Email volume matters or delivery must be guaranteed - move to a durable queue (RQ/Celery + Redis, or SQS) with retries and a DLQ. The `EmailProvider` seam doesn't change; only the dispatch does.

13. Pluggable storage and rate-limiting behind ABCs

Context. Resumes need to live somewhere; today that's local disk, in production it's object storage. Rate limiting is in-process today; horizontal scale needs shared state.

Decision. Both sit behind an interface with the local implementation as the default:

- `StorageBackend ABC` → `LocalStorageBackend` (dev, Docker volume) /

`S3StorageBackend` (any S3-compatible store; boto3 calls threadpooled; presigned URLs). Selected by `STORAGE_BACKEND`. Resumes are **only** served through the authenticated endpoint, never a public URL.

- `RateLimiter ABC` → `InMemoryRateLimiter` (fixed window, keyed by

`bucket:client-ip, X-Forwarded-For` aware) / `NullRateLimiter` (tests). Selected in `get_rate_limiter()`.

Consequences.

- The scale-up path is "add one class implementing the interface, change one

selector" - no call-site churn.

- **The in-process rate limiter is per-instance.** Run N replicas and the

effective limit is $N \times$ the configured number, and it resets on deploy. Correct for a single instance; a `RedisRateLimiter` is the documented fix.

- Fixed-window (not sliding-window / token-bucket) allows a $2 \times$ burst at a

window boundary. Fine for the abuse-prevention goal here.

Revisit when. We run more than one API instance - the rate limiter needs Redis before that's true, or it's not really enforcing anything.

14. Frontend: URL-as-state, server components, no data-fetching library

Context. The dashboard has filter, search, and pagination. The classic approach is client components + React Query / SWR + a client cache to keep in sync with the server.

Decision. List views are **Server Components**. Filter/search/page live in the URL query string (`?state=&search=&page=`); navigation re-renders the server component with fresh data. No client-side data store, no fetch-on-mount. The one mutation (state transition) is a **Server Action** behind a confirm dialog, followed by `revalidatePath + router.refresh()`. Column sort - the only truly ephemeral view state - is client-side within the current page. Styling is CSS Modules + design tokens, no CSS framework.

Consequences.

- No client/server cache to reconcile; views are shareable and

back-button-correct by construction; less JS shipped.

- The token stays server-side (ADR 1) because the fetch is server-side.
- Every filter change is a server round-trip - no optimistic client filtering.

At lead volume this is imperceptible; for large lists it would feel slow.

- No live updates - two attorneys working simultaneously see each other's

changes on their next navigation/refresh, not instantly.

- Hand-rolled CSS is more upfront work than a utility framework; the payoff is

no framework lock-in on a small, bespoke UI.

Revisit if. The list grows past a few thousand rows (add server-side keyset pagination + maybe client filtering of the current page), or real-time collaboration matters (SSE on the activity feed is the smallest step).

15. Confirm dialogs instead of an undo/toast pattern

Context. Marking a lead reached out, and reopening one, are consequential actions in a multi-user tool. The two common patterns are **confirm-before** and **act-immediately-with-undo**.

Decision. Confirm-before, via a shared `ConfirmDialog`. Undo is provided at the **domain** level instead the transition is reversible (ADR 7) - rather than as a transient UI affordance.

Rationale. A toast-undo needs a grace period during which the change is pending or silently reversible; in a tool where a second attorney might already be looking at that lead, "pending for 5 seconds" is ambiguous state. An explicit confirm is unambiguous, and because the action is genuinely reversible in the domain, a misclick that gets past the dialog is still recoverable - with an audit entry, which a toast-undo wouldn't leave.

Consequences.

- No ambiguous in-between state; every committed change is intentional and

logged.

- A second click on every transition. Acceptable for low-frequency, high-stakes

actions; it would be friction on a bulk workflow.

Revisit if. Attorneys need to action many leads in a sitting - then bulk select + one confirm for the batch, or an undo window on the batch.

Cross-cutting non-goals (deliberately out of scope)

Not built	Why it's acceptable to omit here	What it would take
Email verification on registration	Demo; but it's the precondition for ADR 5 in production	Token mail + <code>email_verified</code> flag + gate /my
Token revocation / refresh rotation	Small trusted user set, short-lived demo	Short TTL + refresh + optional <code>jti</code> denylist
Durable email delivery	Low volume, fails safe	Queue + retries + DLQ behind the existing <code>EmailProvider</code>
Virus scanning of uploads	Attorneys download to review; allow-list + sniff limits the surface	ClamAV / a scanning API in the upload path
Distributed rate limiting	Single instance	<code>RedisRateLimiter</code> implementing the existing ABC
Full-text / ranked lead search	<code>ILIKE</code> on name/email is enough at this scale	Postgres FTS or a search index
RBAC beyond attorney / applicant	One internal persona in the brief	<code>role</code> column + per-transition rules (ADR 6)
Live dashboard updates	Refresh-on-navigation is fine for the team size	SSE on <code>/leads/activity</code>